

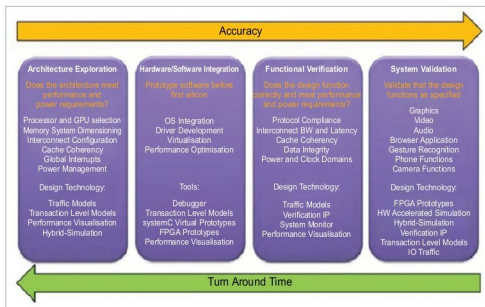


Fig. 4: Hybrid emulation

sors. Typical results might show the impact on performance of variations in the number of data streams and processing loads.

Early software development and software-driven verification. Hybrid emulation enables pre-silicon software development while part of the RTL design is still under development. It is also useful for system-level hardware-software co-verification. Virtual prototypes support the use of instruction-accurate SystemC/C++ processor models and other loosely-timed SystemC TLM-2.0 models. Such models remove the need to wait for silicon to begin developing or validating system software.

In the example given in Fig. 4, a video-processing system running in emulation is debugged with image decompression and LCD driver software running on a virtual prototype, displaying real-time video output on virtual display.

The software team can leverage these functional models in a virtual prototype developed using a virtualiser, and the resulting software development kit is called a virtualiser development kit, with some pieces of the design for which no SystemC models exist. These are represented as RTL in a high-performance emulator like ZeBu Server-3.

This hybrid emulation environment allows the software design team to begin developing, running and testing its software at multi-megahertz speeds long before silicon is available. The hybrid emulation mode is also valuable for software-driven verification when the RTL for the processors is not available or when you wish to free up some of the emulation system capacity by off-loading processor emulation models and other blocks of the design to the virtual prototype.

In the hybrid emulation environment, while the processor models are executing software in the virtual prototype, the emulator also provides full visibility into the design's RTL to allow simultaneous verification of the non-processor design elements. In such environments, typically, embedded software performs as an executable test bench in combination with emulation focused

transaction-level tests using virtual components, C++ test benches, software traffic/packet generators or physical testers.

To achieve the hybrid emulation speed required for software development and testing within the virtual prototype, several parameters are not modelled with 100 per cent accuracy in a virtual prototype, such as detailed timing. Consequently, synchronisation mode for hybrid emulation for software development is generally loosely-timed or approximately-timed, which can still provide the accuracy level needed for software development as well as the performance expected.

A SystemC loosely-timed processor model must describe processor behaviour accurately enough to execute real, unmodified software images, and model how the system would boot the operating system, support actual drivers and run applications.

Software development or software-driven verification. In using hybrid emulation for software development or software-driven verification, the processors are modelled in the virtual prototype using SystemC loosely-timed models such as ARM Fast Models, while major pieces of the design may be modelled in the emulator. The processors and sub-systems are connected to emulated RTL blocks through transactors for bus-level communication with TLM-2.0 wrapper or adaptor. Software debug is accomplished using traditional software development tools such as ARM DS-5, Lauterbach Trace32 or inherent tools of the virtual prototype environment.

For software development teams, hybrid emulation enables you to perform multi-core software debug in a very high-performance environment without waiting for silicon or transaction-level models to be available. For verification teams, hybrid emulation enables high-performance software-driven verification even when processor RTL is not available for emulation, while freeing emulator



Fig. 5: ZeBu server